



SC4003 CE/CZ4046

INTELLIGENT AGENTS

Assignment 1

Amudhan Anandharaman

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

NANYANG TECHNOLOGICAL UNIVERSITY

Table of contents

1. Introduction	3
2. Value iteration	4
2.1 Optimal Policy	4
2.2 Utility Values	5
2.3 Utility estimates as function of iteration number	6
3. Policy Iteration	7
3.1 Optimal policy	8
3.2 Utility values	9
3.3 Utility Estimates as a function of iteration number	10
4. Part 2: Complex maze Value and Policy iteration	11
4.1 Design	11
4.2 Results on the larger array	12
Results for value iteration:	12
Results for policy iteration	15
4.3 Effect of number of states and environment complexity on convergence	17
5. Conclusion	21
6. References	22

1. Introduction

This report models the given maze (seen below) as Markov Decision Process and solves it using value iteration and policy iteration. The agent can move up, down, left or right, with stochastic transitions. The intended action occurs with probability of 0.8 while the other right angle outcomes occur with 0.1 probability.. White cells have a reward of -0.05 and green cells have a reward of 1 while brown has a reward of -1. Since there are no terminal states, the process will continue infinitely. A discount factor of 0.99 is used as the assignment states.



Figure 1.1 - given in assignment

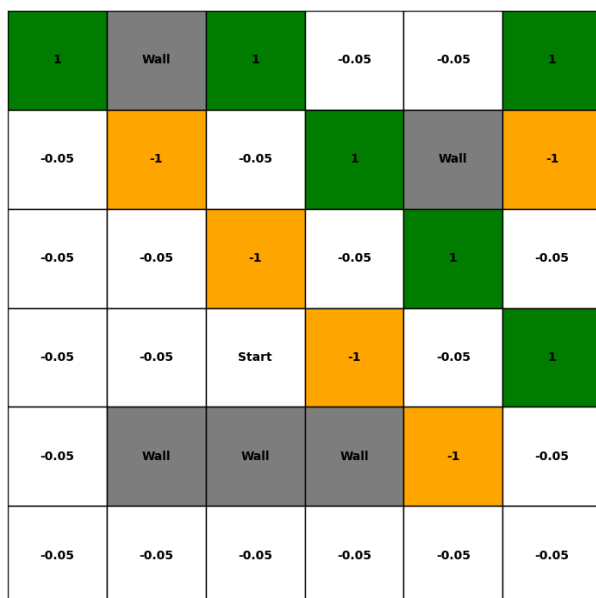


Figure 1.2 - The grid visualised using matplotlib in python

The objective is to compute the utility of each non wall state, determine the optimal policy and compare the behaviour of both algorithms on the original maze and on a more complex maze design in part 2

2. Value iteration

Value iteration was implemented by initializing the utility of every non-wall state to 0 and repeatedly updating it using the bellman equation.

$$U_{k+1}(s) = R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) U_k(s')$$

A discount factor of 0.99 is used as stated by the assignment. This is how the above equation looks like in the code

```
utility_new[x][y] = rewards[x][y] + DISCOUNT_FACTOR * highest_utility_from_directions
```

For this assignment I have decided to use the synchronous approach, which means all node utilities were calculated using the old utility array instead of continuously updating one utility array. For each state, the expected utility of all four actions was calculated under the given stochastic transition model, and the action with the highest expected utility was used for the update. If an action would move the agent to a wall or outside the grid, the agent will remain at the same state. The iteration continues until the max change in utility across all non-wall states falls below a threshold. For this case, I decided on a convergence threshold of 0.001.

2.1 Optimal Policy

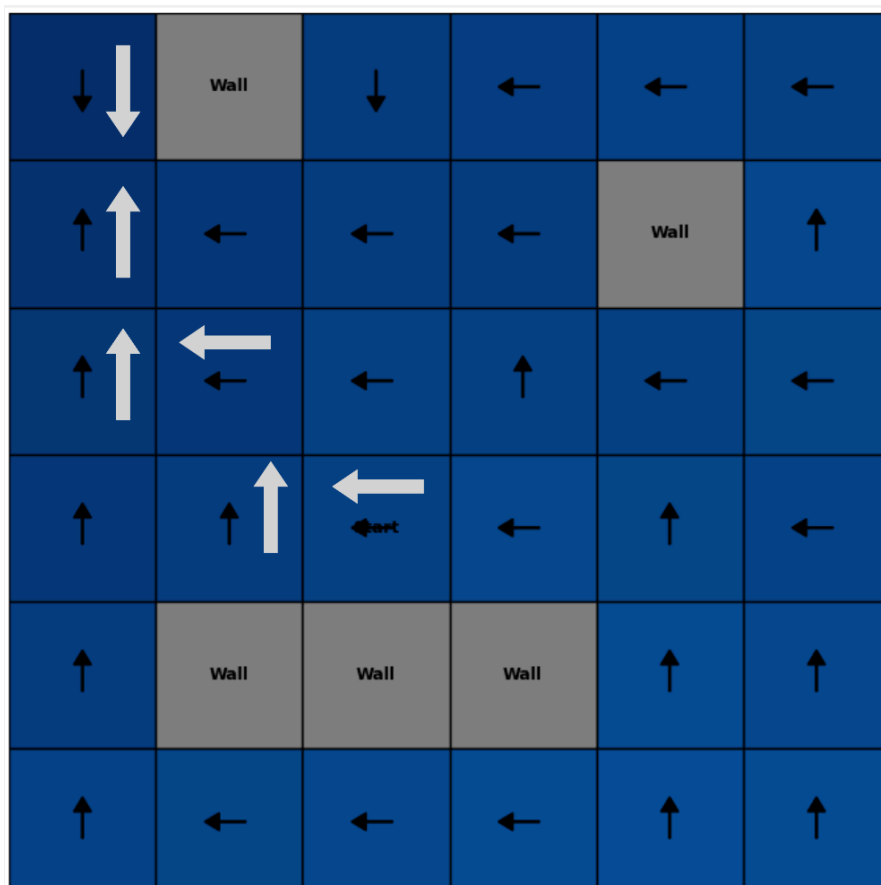


Figure 2.1 - figure of the final policy

The optimal policy obtained from value iteration is shown in Figure 2.1. It converged after 689 iterations. The white arrow shows what the agent will do if it starts from the start. As can be seen, the agent will go to the top left and loop there forever as the maze has no terminal state. The policy does not necessarily direct the agent to a single goal and stop. Instead it maximises the long run expected rewards, which can lead to repeated movement between 2 states in a high value region.

2.2 Utility Values

99.902	Wall	94.921	93.739	92.507	93.185
98.282	95.769	94.418	94.269	Wall	90.775
96.824	95.451	93.158	93.036	92.961	91.643
95.416	94.304	Start 93.075	90.972	91.662	91.737
94.162	Wall	Wall	Wall	89.396	90.405
92.774	91.554	90.349	89.158	88.401	89.125

Figure 2.2 - utility values of all nodes after 689 iterations

These are the final values of all the non wall states. These utilities represent the long run discounted returns obtained by starting from each state. Because there are no terminal states, the agent continues moving indefinitely, so the utility value becomes large and larger than the immediate reward of any cell. This differs from textbook terminal state gridworld examples as usually those are terminal states and have fixed end points

2.3 Utility estimates as function of iteration number

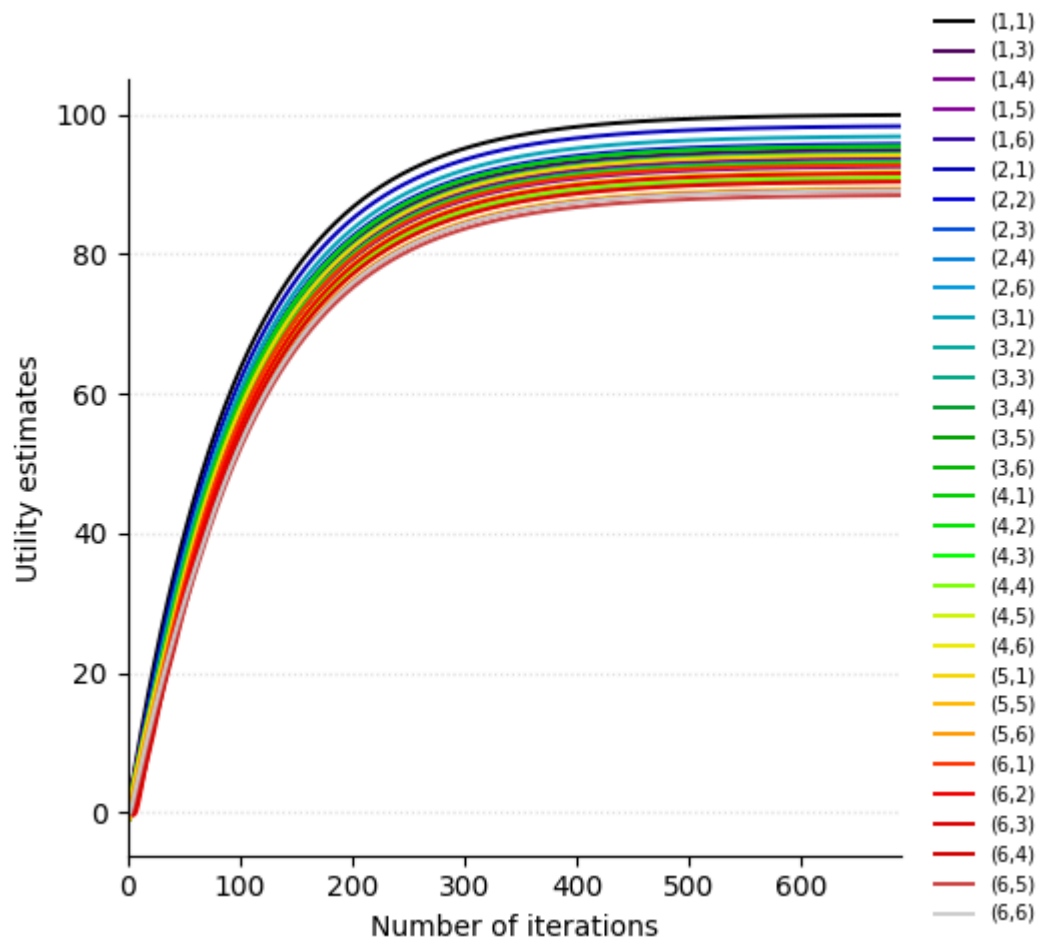


Figure 2.3 - Utility estimates as function of iteration number

Using the graph as a sample from “Artificial Intelligence: A Modern Approach” by S. Russell Figure 17.5(a) [1], we change the lines from black and white and dotted to solid lines and coloured to have a better visualisation. The utility estimate graph for value iteration is shown above in figure 2.3. Since all non-wall states were initialised to 0, the utility values changed fast during the early iterations as the utility “spread” throughout the maze. The curves eventually flattened as the maximum absolute change between successive utility tables fell below the threshold of 0.001. The below code shows this logic.

```
diff = abs(utility_new[x][y] - utility_old[x][y])
delta = max(delta, diff)
```

```
if delta < THRESHOLD:
    print(f"Converged after {iteration + 1} iterations.")
    break
```

3. Policy Iteration

Policy iteration is similar to value iteration but with one additional step, policy improvement. Instead of directly selecting the best action during every utility update, policy iterations begins with an initial policy and repeatedly improves it until it becomes stable. At the start of the algorithm, all of the policy is set to “up”. The algorithm then enters the outer loop, which repeatedly alternates between evaluating the current policy and improving it.

```
#initial policy for all non-wall states. starts
policy = [
    ["up", "wall", "up", "up", "up", "up"],
    ["up", "up", "up", "up", "wall", "up"],
    ["up", "up", "up", "up", "up", "up"],
    ["up", "up", "up", "up", "up", "up"],
    ["up", "wall", "wall", "wall", "up", "up"],
    ["up", "up", "up", "up", "up", "up"],
]
```

First we talk about the policy evaluation. In this stage the policy is kept fixed, and the utility of each non-wall state is repeatedly updated according to the action assigned by the policy. Unlike value iteration, this step does not take into account all four actions. Instead it just takes the policy and calculates the utility from that specific action. This is the updated formula

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' | s, \pi(s)) U^{\pi}(s')$$

And in code form (policy evaluation)

```
#update the utility for the current state using the value iteration formula
utility_new[x][y] = rewards[x][y] + DISCOUNT_FACTOR * utility_from_action
```

This policy evaluation step was implemented using an inner loop which repeatedly updated the utility table until the maximum absolute change between new and old utility tables fell below the convergence threshold of 0.001

The second stage is policy improvement. Once the utility for the current policy converges, the algorithm examines each non-wall state and tests all four possible actions. For each action the expected utility is calculated using the same stochastic transition model. The action with the highest utility will then be selected and put into the policy as the new policy for that state. After this, the policy is compared to the old policy, and if it is the same, policy iterations stop. If it is not the same, the outer loop is repeated i.e., the policy evaluation.

```

for direction in ["up", "down", "left", "right"]:
    expected_utility = get_expected_utility_for_direction(
        utility_old, possible_directions, x, y, direction
    )
    if expected_utility > best_utility:
        best_utility = expected_utility
        best_direction = direction

```

Above is the code for the policy improvement. If there is a better policy, the best direction gets updated and that gets assigned to the policy.

```

policy[x][y] = best_direction

```

3.1 Optimal policy

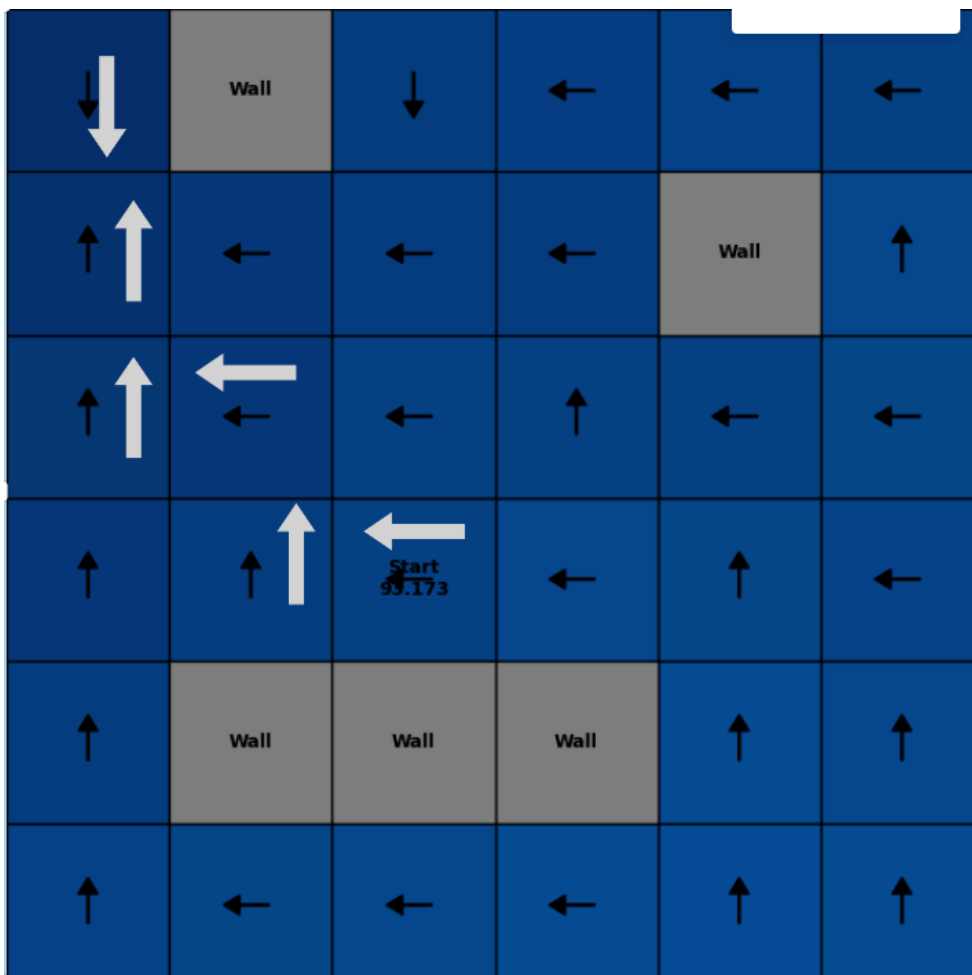


Figure 3.1 - Final optimal policy using policy iteration

The optimal policy obtained from policy iteration is shown above in figure 3.1. Similar to value iteration, the final policy directs the agent towards the top left and starts looping at that spot. Since the environment contains no terminal state, the agent does not stop and keeps looping. The final policy

obtained from policy iteration is exactly the same as value iteration which is consistent with both methods converging to the same optimal policy.

3.2 Utility values

Utility table

100.000	Wall	95.019	93.838	92.605	93.283
98.380	95.868	94.516	94.367	Wall	90.873
96.922	95.549	93.256	93.134	93.059	91.741
95.514	94.402	Start 93.173	91.070	91.760	91.835
94.261	Wall	Wall	Wall	89.494	90.503
92.872	91.652	90.447	89.257	88.498	89.222

Figure 3.2 - Final utility value of each node

Figure 3.2 shows the final utility values of all non wall states after policy iteration converged. These values represent the long run discounted return obtained by starting in each state and following the final stable policy. Since there are no terminal nodes, the utility values are not limited to the immediate rewards of the individual cells. Instead, they reflect the cumulative effect of long run discounted rewards over an infinite sequence of moves.

3.3 Utility Estimates as a function of iteration number

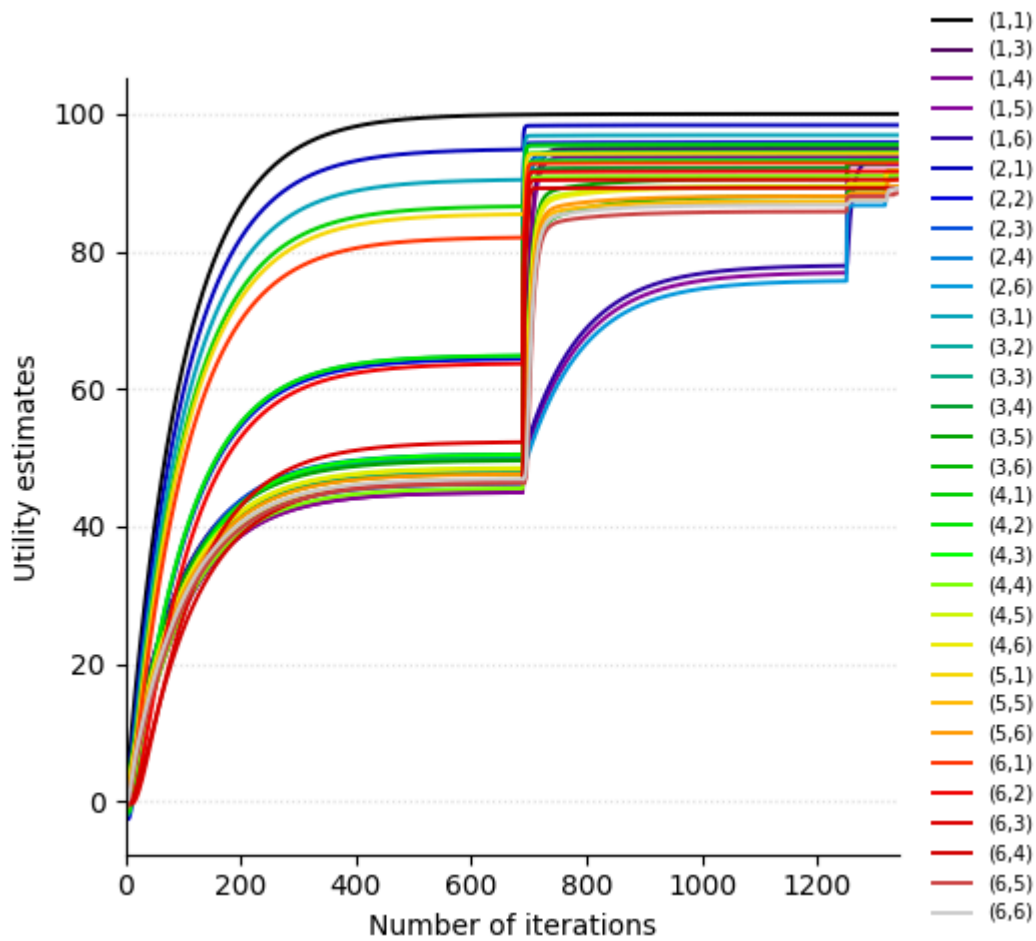


Figure 3.3 - Utility estimates as a function of iteration number of policy iteration

Because of the two stages, policy evaluation and policy improvement, the graph does not show a single smooth convergence. Instead the utility curves first rise and flatten during the policy evaluation, then jumps when the policy is improved. It then continues to converge again under the updated policy. This sequence repeats until the policy becomes the same as the old policy. At which point the utility estimates also stabilise

4. Part 2: Complex maze Value and Policy iteration

4.1 Design

For part 2, I designed a more complex maze environment and tested using the same value iteration and policy iteration from part 1. The custom maze used a 10x10 grid, included more walls, more penalty states and 3 reward regions. This was intended to increase the number of states and the structural complexity of the environment.

```
def get_big_rewards_grid():
    return [
        [-0.05, -0.05, "wall", -0.05, -0.05, -0.05, "wall", -0.05, 1, -0.05],
        ["wall", -0.05, "wall", -0.05, "wall", -1, "wall", -0.05, "wall", -0.05],
        [-0.05, -0.05, -1, -0.05, "wall", -1, -1, -0.05, "wall", -0.05],
        [-0.05, "wall", -1, -1, "wall", "wall", "wall", -0.05, "wall", -1],
        [-0.05, -0.05, -0.05, -0.05, -0.05, -0.05, "wall", -0.05, -0.05, -0.05],
        ["wall", "wall", "wall", "wall", "wall", -0.05, "wall", "wall", "wall", -0.05],
        [-0.05, -0.05, -0.05, -0.05, "wall", -0.05, -0.05, -0.05, "wall", -0.05],
        [-0.05, "wall", "wall", -0.05, "wall", "wall", "wall", -0.05, "wall", -0.05],
        [-0.05, -0.05, "wall", -0.05, -0.05, -0.05, "wall", -0.05, -0.05, -0.05],
        [1, -0.05, "wall", "wall", "wall", -0.05, -0.05, -0.05, "wall", 1],
    ]
```

Figure 4.1 - reward array used

Start	-0.05	Wall	-0.05	-0.05	-0.05	Wall	-0.05	1	-0.05
Wall	-0.05	Wall	-0.05	Wall	-1	Wall	-0.05	Wall	-0.05
-0.05	-0.05	-1	-0.05	Wall	-1	-1	-0.05	Wall	-0.05
-0.05	Wall	-1	-1	Wall	Wall	Wall	-0.05	Wall	-1
-0.05	-0.05	-0.05	-0.05	-0.05	-0.05	Wall	-0.05	-0.05	-0.05
Wall	Wall	Wall	Wall	Wall	-0.05	Wall	Wall	Wall	-0.05
-0.05	-0.05	-0.05	-0.05	Wall	-0.05	-0.05	-0.05	Wall	-0.05
-0.05	Wall	Wall	-0.05	Wall	Wall	Wall	-0.05	Wall	-0.05
-0.05	-0.05	Wall	-0.05	-0.05	-0.05	Wall	-0.05	-0.05	-0.05
1	-0.05	Wall	Wall	Wall	-0.05	-0.05	-0.05	Wall	1

Figure 4.2 - Maze visualised

Compared to the original maze, this contains more walls and more obvious routes. The arrangement of walls allows for multiple possible paths and a risk reward trade off for the agent to decide. In particular, shortcuts were deliberately discouraged by placing brown squares near more direct routes, so that the agent would have to weigh a shorter but riskier path against a longer and safer path.

4.2 Results on the larger array

Both value iteration and policy iteration were rerun on the custom maze. In both cases, the algorithm converged to a stable utility function and policy. The final policy still directed the agent towards a reward that had the most walls around it. Since the agent would stay at the same spot if there were many walls, the utility of that node will be the highest.

Results for value iteration:

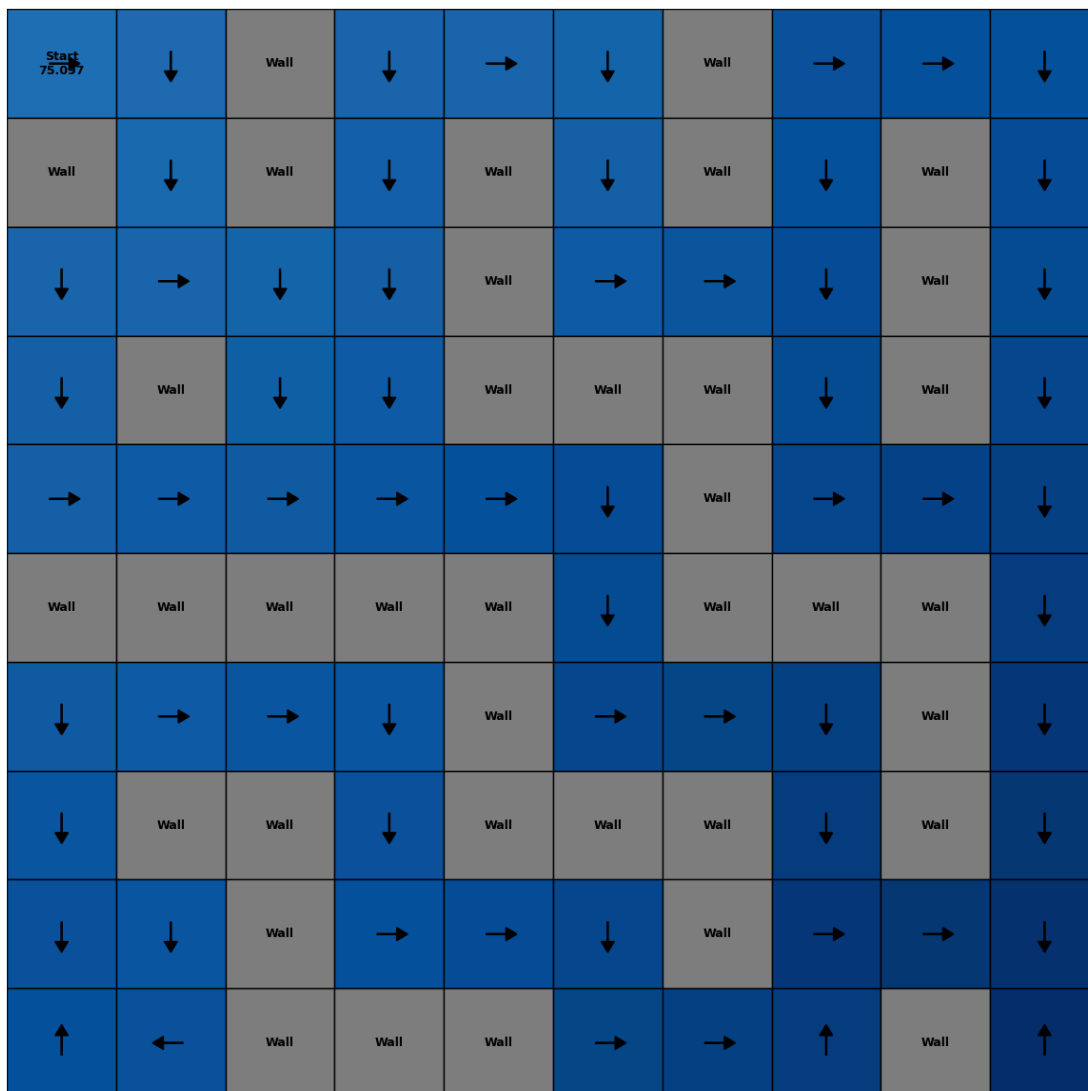


Figure 4.3 - Final optimal policy for the custom maze

Start 75.097	76.109	Wall	79.193	78.486	79.542	Wall	86.346	87.442	87.281
Wall	77.261	Wall	80.346	Wall	80.742	Wall	86.808	Wall	88.427
79.219	78.301	79.484	81.425	Wall	83.026	85.623	87.968	Wall	89.608
80.398	Wall	81.657	82.760	Wall	Wall	Wall	89.437	Wall	90.804
81.477	82.706	83.814	85.206	86.652	87.811	Wall	90.630	91.988	93.214
Wall	Wall	Wall	Wall	Wall	89.129	Wall	Wall	Wall	94.608
83.659	82.975	84.087	85.213	Wall	90.319	91.672	92.894	Wall	95.867
84.865	Wall	Wall	86.494	Wall	Wall	Wall	94.284	Wall	97.142
86.001	84.990	Wall	87.651	88.966	90.154	Wall	95.539	97.142	98.433
87.277	86.001	Wall	Wall	Wall	91.505	92.894	94.131	Wall	99.902

Figure 4.4 - Final utility values of the custom maze

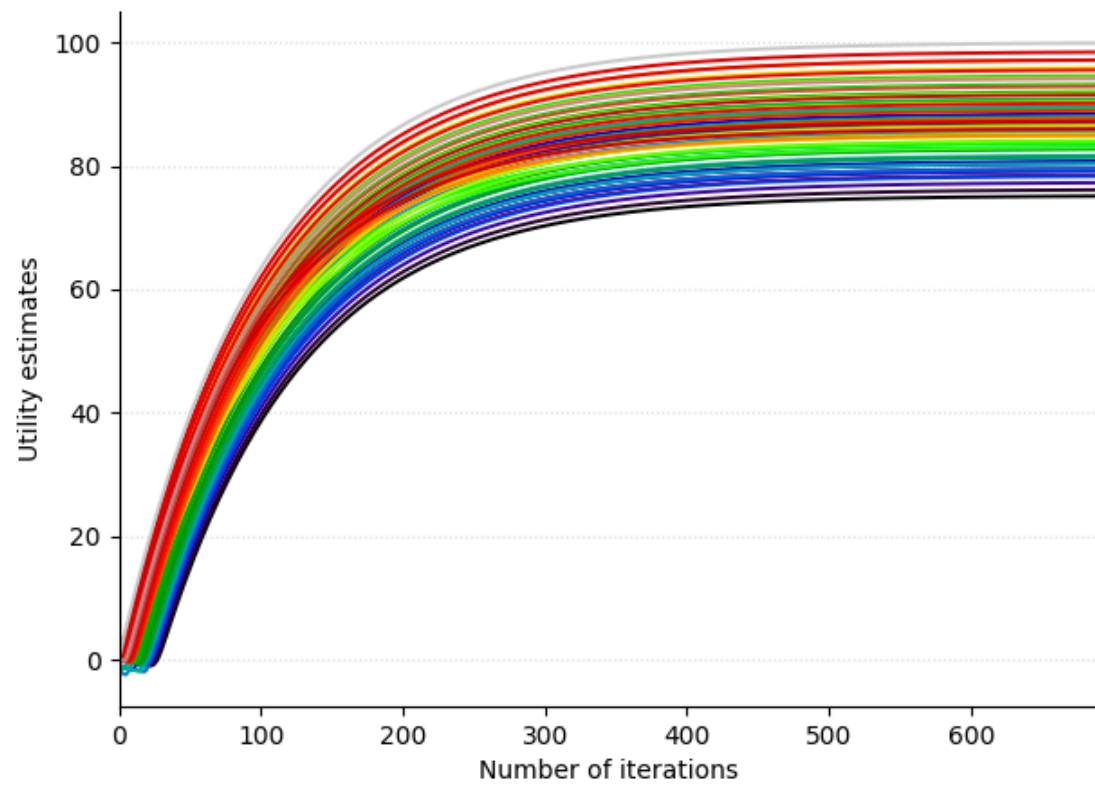


Figure 4.5 - Utility estimates as a function of iteration number for the maze

Value iteration took the same number of iterations with the same values. 689 iterations.

Results for policy iteration

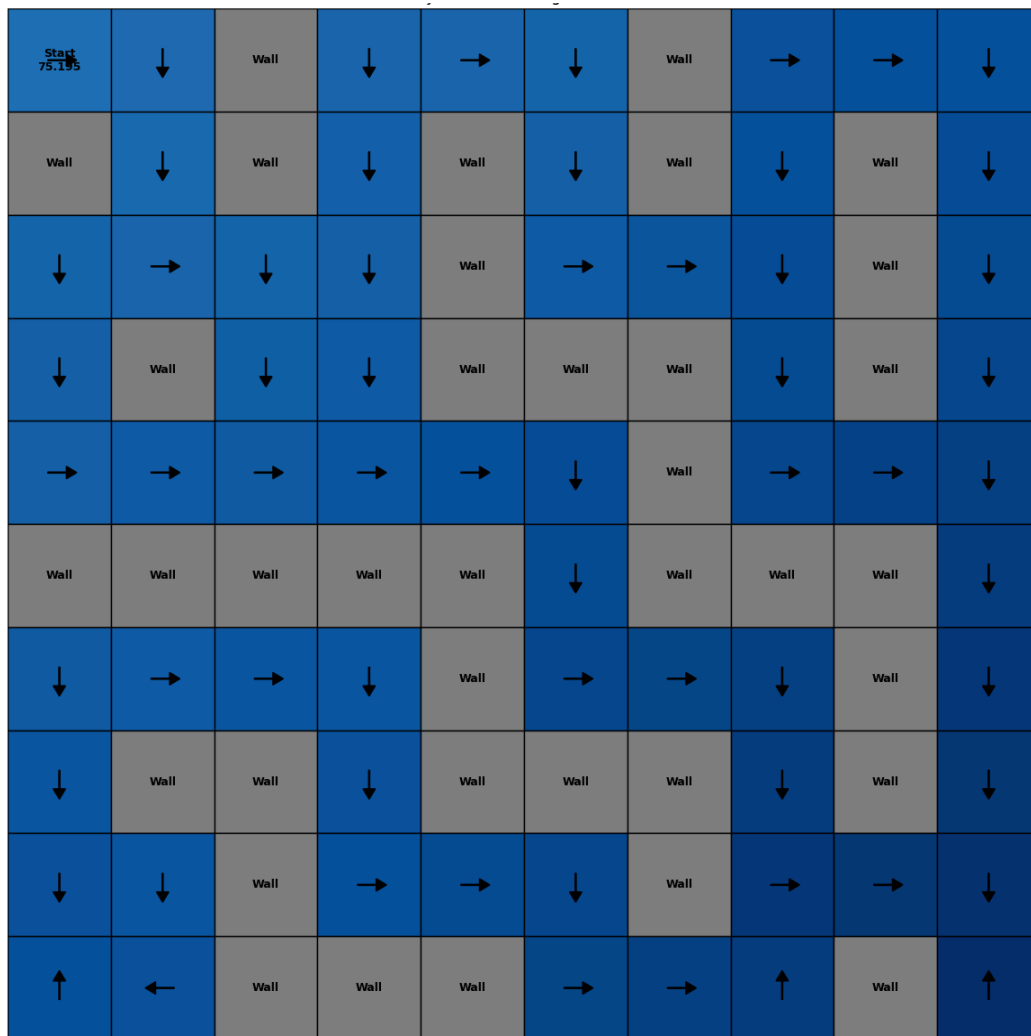


Figure 4.6 - Final optimal policy for the custom maze

Start 75.195	76.208	Wall	79.291	78.584	79.640	Wall	86.443	87.539	87.379
Wall	77.360	Wall	80.444	Wall	80.841	Wall	86.906	Wall	88.525
79.317	78.400	79.583	81.523	Wall	83.124	85.722	88.067	Wall	89.706
80.496	Wall	81.755	82.858	Wall	Wall	Wall	89.535	Wall	90.902
81.576	82.804	83.912	85.305	86.751	87.909	Wall	90.728	92.086	93.312
Wall	Wall	Wall	Wall	Wall	89.227	Wall	Wall	Wall	94.707
83.746	83.073	84.185	85.312	Wall	90.417	91.770	92.992	Wall	95.966
84.951	Wall	Wall	86.593	Wall	Wall	Wall	94.382	Wall	97.240
86.087	85.076	Wall	87.749	89.065	90.252	Wall	95.637	97.240	98.531
87.363	86.087	Wall	Wall	Wall	91.604	92.992	94.229	Wall	100.000

Figure 4.7 - Final utility values of the custom maze

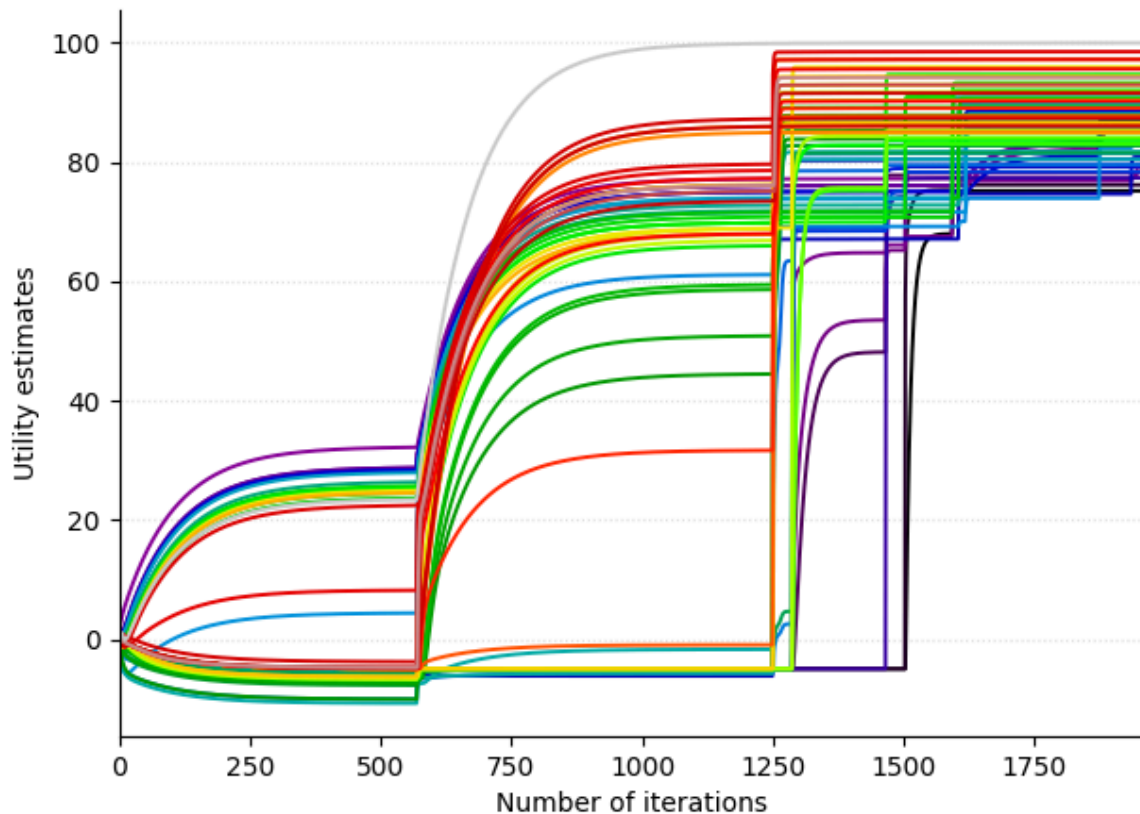


Figure 4.8 - Utility estimates as a function of iteration number for the maze

For policy iteration, the more complex maze required a greater number of iterations before convergence. In addition, the number of times policy evaluation had to be repeated before the policy became stable was also higher than in the original maze

Although the maze was designed to make shortcuts less attractive, the learned policy still preferred a relatively direct route towards the bottom right rather than taking the alternative part. This shows that the agent still preferred the shorter route, which reached the profitable region more quickly rather than the longer path with less brown nodes. This is true for both value and policy iteration indicating that both algorithms converged to the same solution in the large environment.

4.3 Effect of number of states and environment complexity on convergence

From the experiments, the more complex maze had an effect on policy iteration but no effect on the value iteration. Policy iteration required more iterations before convergence and the number of times policy evaluation had to be repeated was also higher. To make sure this is not an edge case, I randomly generate a 30x30 grid to test the consistency of this statement.

```
def get_random_big_rewards_grid(rows=30, cols=30, choices=None, weights=None, seed=None):
    if choices is None:
        choices = [1, -0.05, -1, "wall"]
    if weights is None:
        weights = [1, 8, 1, 4]

    rng = random.Random(seed)
    return [rng.choices(choices, weights=weights, k=cols) for _ in range(rows)]
```

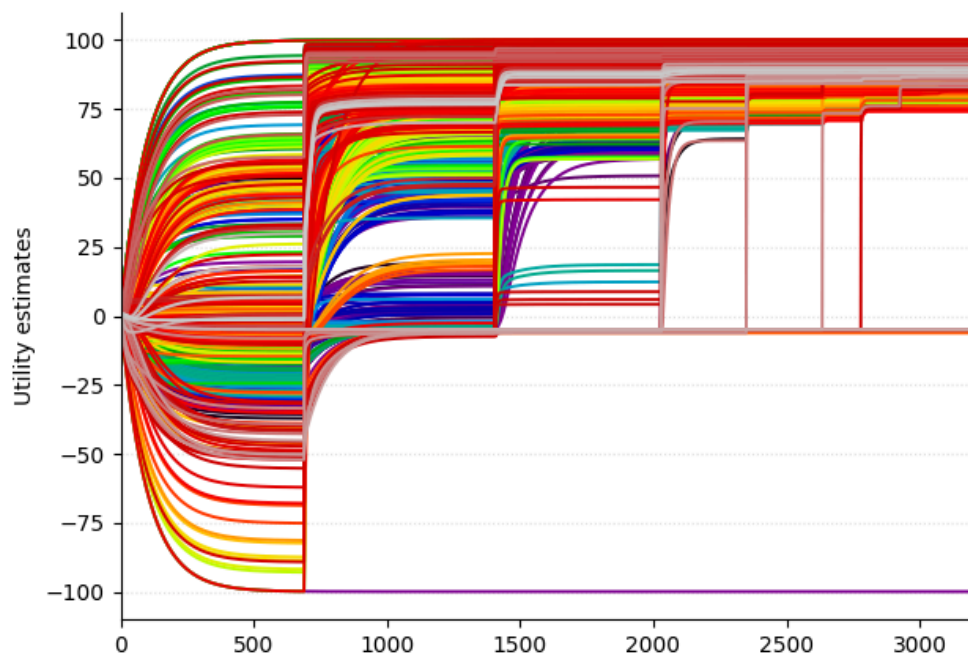


Figure 4.9 - Utility estimates of 30x30 grid as a function of iteration number for the maze

The number of iterations increased with the 30x30 grid but the number of convergences did not seem to be affected by the size of the grid. This suggests that increasing the number of states can increase the amount of computation required, while not necessarily increasing the number of policy updates needed before convergence.

I also tested a 100x100 grid just to see if both value and policy iteration can still learn the right policy. The time taken to run and generate the graphs were significantly higher. As the number of nodes increases in n^2 .

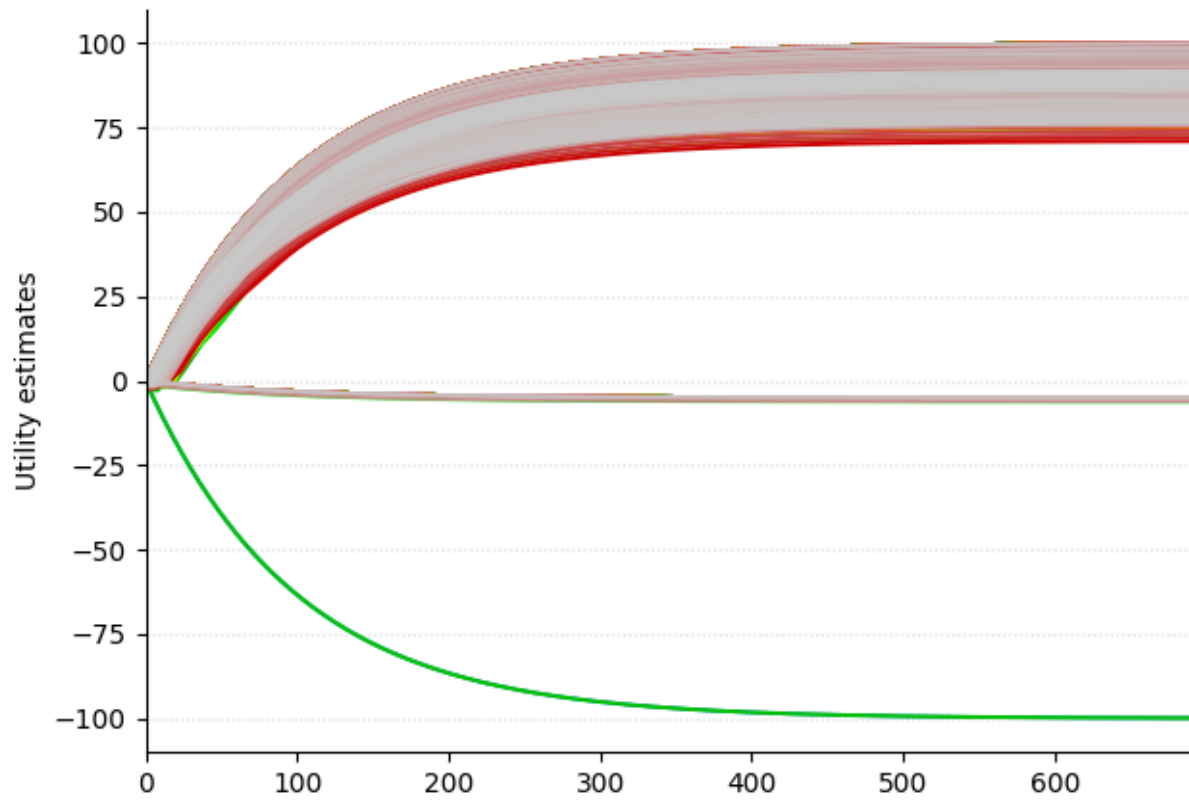


Figure 4.10 - Utility estimates of 100x100 grid as a function of iteration number for the maze (Value iteration)

The value iteration still only took 689 iterations

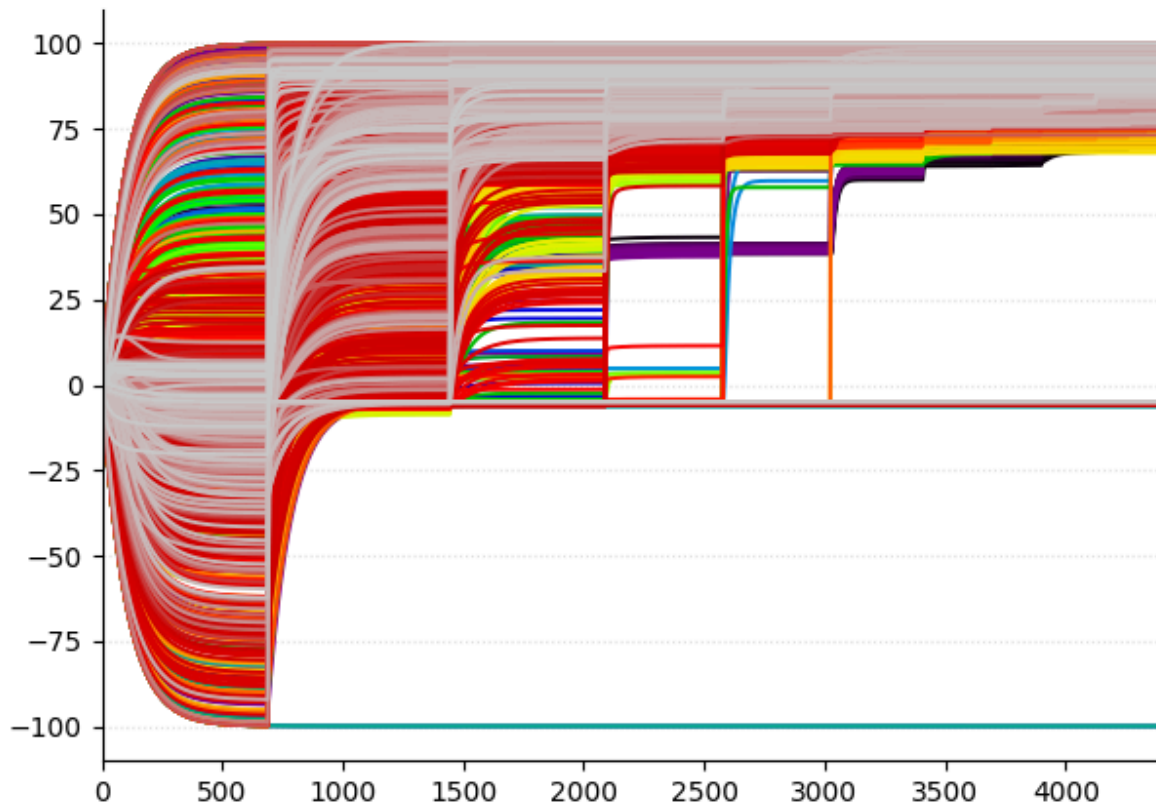


Figure 4.11 - Utility estimates of 100x100 grid as a function of iteration number for the maze (Policy iteration)

The results suggest that the environment can be made significantly more complex while still allowing the correct policy to be learned. In the project, even the large test environments such as 30 x 30 and 100 x 100 grids were still able to converge to a stable policy that matched the reward structure of the maze. Therefore, the practical limit was not if the algorithms could learn the correct policy, but how much time was needed for computation. As the environment becomes larger and more complex, the algorithms remain complete but the runtime increases.

5. Conclusion

In part 1, both value and policy iteration were successfully implemented using the given transition model, reward function and discount factor of 0.99. For the original maze, both algorithms were able to compute the utility values of all non-wall states and produce a stable optimal policy. The final policy and utility values were displayed for both methods and utility estimates were plotted as a function of iteration number. The value iteration graph showed a gradual convergence process, while the policy iteration graph showed a staged pattern caused by the repeated cycles of policy evaluation and improvement. In both cases, the algorithms converged to a consistent solution.

In part 2 a more complex maze environment was designed and the same algorithms were applied again. The larger maze increases both the number of states and the structural complexity of the environment by adding more walls, more constraint routes and competing reward paths. The results showed that complexity affected the two algorithms differently. Policy iterations required more iterations while value iterations just stayed at 689 iterations with a threshold of 0.001.

To answer the question of how complex the environment can become while still learning the correct policy, the experiments showed that the algorithms were still able to learn a stable and sensible policy even when the maze size was increased beyond the custom environment to 30x30 and 100x100 grids. This means the environment can be made more complex than the original maze and still be complete. The main limitation was how much computation was required before convergence. As the grid became larger, the computation cost was significantly increased but the algorithms were still able to converge successfully.

Overall the results show that both value iteration and policy iteration are effective methods for solving the maze mdp problem. The original maze and the more complex maze were both solved successfully. And the experiments showed that the number of iterations for policy iteration increases as the size of the maze increases, while value iterations stays the same. In many of the mazes, the learned policy tended to move the agent toward a corner region containing a positive reward. This is because once the agent is near a rewarding corner, attempted moves into walls cause it to remain in the same state which increases the expected long run utility of staying in the region. As a result, the corner reward state can become especially attractive under the given transition model. It would also have been interesting to test the same environment with terminal rewards states since that would change this behaviour significantly. Instead of looping in a high value region indefinitely, the agent would be forced to end the episode on reaching a terminal state, leading to a different policy and of much smaller utility values.

6. References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Harlow, England: Pearson, 2021.